

Объектно- Ориентированное Программирование

(МИСиС, зима-весна 2026)

Полевой Дмитрий Валерьевич

к.т.н., доцент КиК

teams: 8url2s2

e-mail: dvpsun@gmail.com

tg: @dvpsun

что такое «текст» при
разработке ПО?

Где работаем с текстом

- интерфейс
 - графический пользовательский (GUI)
 - командной строки (CLI)
- URI (пути файловой системы, URL-адреса)
- обработка данных

Аспекты работы с текстом

- хранение (долгосрочное)
- передача
- манипулирование

ОСНОВНЫЕ ПОНЯТИЯ

- **ТЕКСТ**
- **СТРОКА**
- **СИМВОЛ**

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Pellentesque id risus eget purus convallis consequat. Nunc convallis sit amet urna sed gravida. Curabitur eu dui vehicula, bibendum massa vitae, ultricies lacus. Ut viverra pellentesque nisi, sit amet accumsan mi interdum sed. Curabitur auctor feugiat elit, in consequat quam interdum vitae. Nam ex mauris, eleifend in felis nec, dignissim cursus tortor. Phasellus consectetur ultricies erat et congue. Quisque mauris leo, vulputate sed nisi at, viverra pulvinar enim.

Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Nullam

Базовые операции с текстом

- расчет количества символов
- обращение к символу
- вставка/удаление/замена подстроки
- поиск подстроки
- изменение регистра

и т.д.

Что такое символ?

Символ

- **образ** - графическое начертание
- **код** - числовое значение (хранение в памяти)
- схема кодирования (кодировочная таблица)

Нет смысла в строке если вы не знаете её кодировки...

Не существует такой вещи как простой текст.

<http://www.joelonsoftware.com/articles/Unicode.html>

Кодовая таблица (ASCII)

b₇ b₆ b₅ b₄ b₃ b₂ b₁ Bits Column Row					0 0 0	0 0 1	0 1 0	0 1 1	1 0 0	1 0 1	1 1 0	1 1 1
b₄ b₃ b₂ b₁	0	1	2	3	4	5	6	7				
0 0 0 0	0	NUL	DLE	SP	0	@	P	`	p			
0 0 0 1	1	SOH	DC1	!	1	A	Q	a	q			
0 0 1 0	2	STX	DC2	"	2	B	R	b	r			
0 0 1 1	3	ETX	DC3	#	3	C	S	c	s			
0 1 0 0	4	EOT	DC4	\$	4	D	T	d	t			
0 1 0 1	5	ENQ	NAK	%	5	E	U	e	u			
0 1 1 0	6	ACK	SYN	&	6	F	V	f	v			
0 1 1 1	7	BEL	ETB	'	7	G	W	g	w			
1 0 0 0	8	BS	CAN	(	8	H	X	h	x			
1 0 0 1	9	HT	EM	)	9	I	Y	i	y			
1 0 1 0	10	LF	SUB	*	:	J	Z	j	z			
1 0 1 1	11	VT	ESC	+	;	K	[	k	{			
1 1 0 0	12	FF	FS	,	<	L	\	l				
1 1 0 1	13	CR	GS	—	=	M	]	m	}			
1 1 1 0	14	SO	RS	.	>	N	^	n	~			
1 1 1 1	15	SI	US	/	?	O	_	o	DEL			

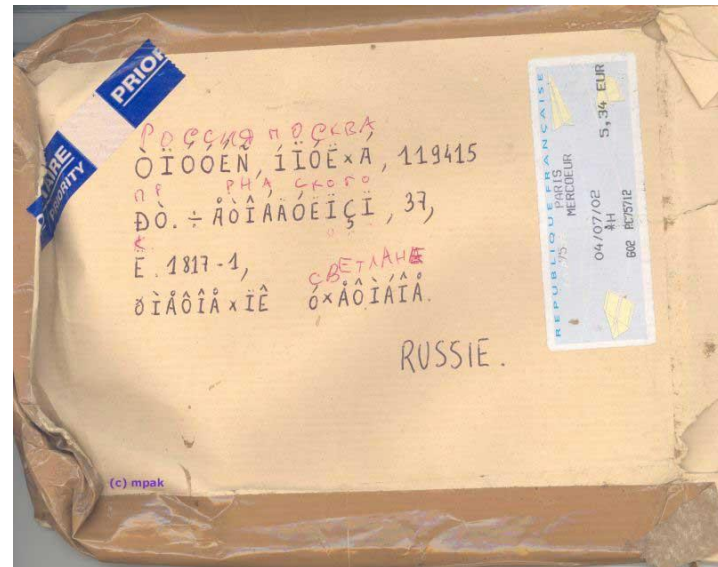
Кодовая таблица (ASCII)

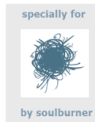
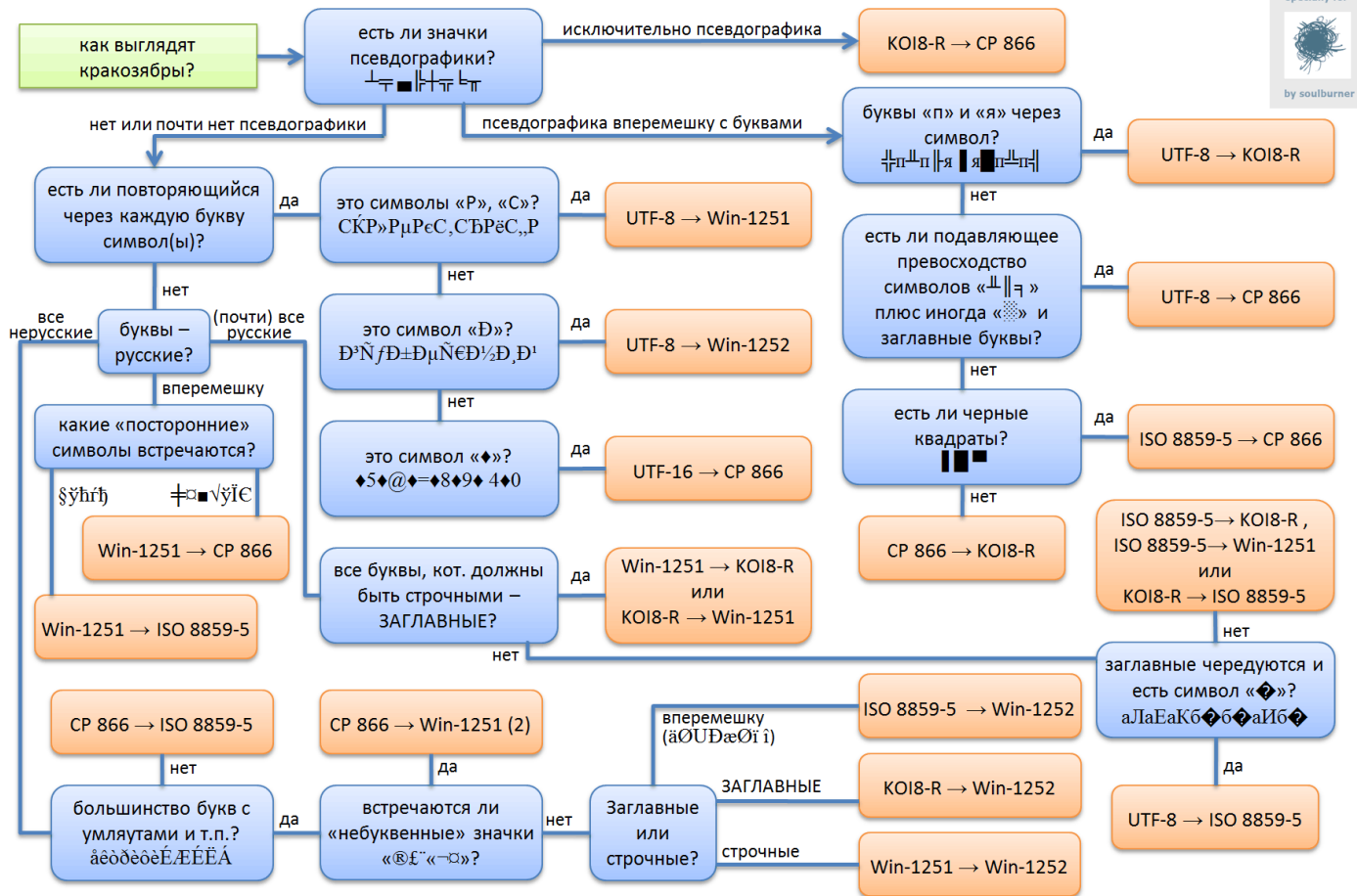
ASCII Code Chart

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2		!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{		}	~	DEL

Кракозябры (крякозябры)

- результат неправильного перекодирования текста
- существуют вариации произношения/написания термина (части *крако-*, *кроко-*, *кряко-*, *карако-*; *-зябры*, *-зябли* и т.п. могут встречаться в разных сочетаниях)





СКОЛЬКО ВСЕГО НАДО СИМВОЛОВ?

Unicode сегодня

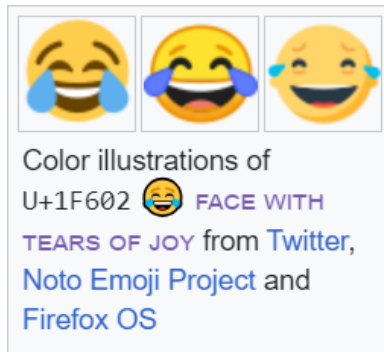
- международный стандарт (версия 15.1)
- универсальный набор символов
- семейство кодировок
- 161 групп (script)
- более 267 000 кодовых точек
- более 149 000 именованных символов
- более 81 000 «иероглифов» (Han)

Например

👑 алхимические средневековые тексты



эмотиконы (emojī)



а еще руны, клинопись, и т.д.

А если без экзотики?

- расскажите это китайцам
- Multilingual European Character Set 2 (MES-2)
1062 символа

Кодовые таблицы и юникод

- национальные (1-байтные) таблицы (CP866, Win1251 и т.д.)
- Unicode (переменный размер)
 - UTF8, UTF16
- Unicode (фиксированный размер)
 - UTF32

Формат кодирования UTF-8

- переменное количество байт (от 1 до 4)
- полная обратная совместимость с ASCII
(для символов U+0000 до U+007F, которые занимают один байт с нулём в старшем бите)
- не зависит от порядка байт системы

UTF-8 (ошибки декодирования)

- недопустимый байт
 - байт продолжения без начального байта
 - отсутствие нужного количества байтов продолжения
- последовательность байтов, декодирующаяся в недопустимую кодовую позицию
- неэкономное кодирование

Формат кодирования UTF-16

- переменное количество байт (2 или 4)
 - суррогатная пара — это две кодовые пары для кодирования одного символа
- порядок байт системы имеет значение (LE или BE)

Формат кодирования UTF-32 (UCS-4)

- постоянное количество байт (4)
- прямое представление кодовой позиции символа
- порядок байт системы имеет значение (LE или BE)

Формат кодирования UTF-32 (UCS-4)

- позволяет обращаться по индексу
- не решает проблемы вычисления отображаемой ширины строки
- существуют знаки, которые можно представлять различными кодами, при помощи комбинирующих символов Unicode

например: букву «Й» можно записать

- отдельный символ
- сочетание базового и комбинированного

И+̣ =Й

Алгоритмы нормализации Unicode

- стандарт определяет 4 алгоритма нормализации текста (NFD, NFC, NFKD и NFKC)
- нормализация осуществляется заменой символов на эквивалентные с использованием таблиц и правил
- декомпозиция – замена (разложение) одного символа на несколько составляющих символов
- композиция – замена (соединение) нескольких составляющих символов на один символ.

Маркер последовательности байтов (Byte Order Mark, BOM)

- специальный символ из стандарта Unicode
- может вставляться в начало файла или потока
- служит признаком кодирования Unicode
- указывает косвенно
 - кодировку
 - порядка байтов (endianness)

Byte Order Mark, BOM

(примеры)

- UTF-8 - EFBBBF_{16}
- UTF-16 (BE) - FEFF_{16}
- UTF-16 (LE) - FFFE_{16}
- UTF-32 (BE) - 0000FEFF_{16}
- UTF-32 (LE) - FFFE0000_{16}

ваши вопросы

Как все начиналось (почти...)

Си-строка

- массив символов `char` заканчивающийся нуль-терминатором
- библиотека `<string.h>`
- источник постоянных ошибок

Проблемы манипуляции Си-строками

```
char* msg = "Message: time to do!";  
char buf[BUFSIZE] = "default buffer text\n";  
char* str = (char*)malloc(sizeof(char) *  
    strlen(buf));  
// ...  
free(str);  
// ...
```

Проблемы манипуляции Си-строками

```
char* msg = "Message: time to do!";  
char buf[BUFSIZE] = "default buffer text\n";  
char* str = (char*)malloc(sizeof(char) *  
    strlen(buf));  
// ...  
free(str);  
// ...
```

Проблемы манипуляции с Си-строками

- низкоуровневая работа с массивом
- без учета константности
- отслеживание конца строки
- отслеживание размера буфера
- большое число функций

Символьный литерал

`'S'`

из каких байтов состоит?

Уточнение типа литерала

- файлы исходного кода тоже имеют кодировку – рекомендуется UTF-8
- префиксы

u8 - UTF-8 (`char`)

u - UTF-16 (`char16_t`)

U - UTF-32 (`char32_t`)

L - широкие символы - не использовать (`wchar_t`)

Управляющие последовательности

\'	single quote	\f	form feed - new page
\"	double quote	\n	line feed - new line
\?	question mark	\r	carriage return
\\	backslash	\t	horizontal tab
\a	audible bell	\v	vertical tab
\b	backspace		

Управляющие последовательности

<code>\nnn</code>	octal value
<code>\xnn</code>	hexadecimal value
<code>\unnnnn</code>	universal character name
<code>\Unnnnnnnnnn</code>	universal character name

Строковый литерал

`"текст внутри двойных кавычек"`

из каких байтов состоит?

Строковый литерал

- имеет тип $T[n]$
- помните о кодировке исходного файла
- используйте префиксы и суффиксы
- помните о терминальном нуле
(подставляется автоматически)
- помните о копированиях литералов

Размер строкового литерала

- однобайтные кодировки
 - размер (в байтах) строкового литерала =
число символов плюс 1 для 0-терминатора
- для функций «размера» помнить, учитывается ли терминальный ноль

Сцепление строковых литералов

```
char str[] = "12" "34";
```

или

```
char str[] = "1234";
```

или

```
char str[] = "12\  
34";
```

Неконстантная инициализация

- строковый литерал, как инициализатор `char*` или `wchar_t*`
- разрешена в C99
- устарела в C++98
- удалена в C++11

Полезно знать

- идентичные строковые литералы могут быть объединены в пул для экономии места в исполняемом файле

UTF-8 строковые литералы (пример)

```
// Before C++20
```

```
const char* str1 = u8"Hello World";
```

```
const char* str2 = u8"\U0001F607 is O:-) ";
```

```
// C++20 and later
```

```
const char8_t* u8str1 = u8"Hello World";
```

```
const char8_t* u8str2 = u8"\U0001F607 is O:-) ";
```

UTF-16 строковые литералы (пример)

```
auto s3 = u"hello"; // const char16_t*  
auto s4 = U"hello"; // const char32_t*
```

Необработанный строковый литерал (raw string literal)

- префикс R и пользовательский разделитель
- может комбинироваться с префиксом типа литерала и суффиксом

пример:

R" (\w \\ \w) "

u8R| (fdfdfa) |

ваши вопросы

Библиотека строк

- `std::string`
- `CString`
- `QString`
- `CComBSTR`
- `FBString`

Базовая схема `std::string`

- данные в куче
- переиспользуемый буфер

```
char* data_  
size_t size_  
size_t capacity_
```

Строковый класс (`std::string`)

- стандартная библиотека
- стандарт не определяет конкретный способ хранения строк в памяти
- стандарт допускает реализацию с подсчетом ссылок

так какие символы хранит `std::string`?

Шаблонное объявление `string`

```
template<
    class Ch,
    class Tr = char_traits<Ch>,
    class Al = allocator<Ch>>
class basic_string;

typedef basic_string<char> string;
```


Характеристика типа символа (стандартная библиотека)

класс-свойств

```
template<class Ch>
```

```
struct char_traits;
```

`char_type` – тип символов

`pos_type` – позицию в потоке (целый тип)

Строки с поддержкой Unicode

- `std::u8string` - для `char8_t` (C++20)
- `std::u16string` - для `char16_t`
- `std::u32string` - для `char32_t`
- `std::wstring` - для `wchar_t` (избегать)

std::string литералы

```
//using namespace std::string_literals;
```

```
auto u8str2 = u8"Hello World"s;
```

```
auto str3 = L"hello"s;
```

```
auto str4 = u"hello"s;
```

```
auto str5 = U"hello"s;
```

Создание строки

- пустая строка

пример:

```
string str;
```

- инициализация строковым литералом

пример:

```
string str("misis");
```

- копирование существующей

пример:

```
string str(strOld);
```

Инициализация строки символом

- невозможна инициализация символом

пример:

```
//string( 'a' )
```

- возможна инициализация несколькими экземплярами символа

пример:

```
string( 'a' , cnt)
```

Присвоение

- `assign`
- `operator=`

пример:

```
strFI.assign(str);
```

```
strFI.assign("");
```

```
strFI = str;
```

```
strFI = "test text";
```

Размер строк

- размер буфера
`capacity`
- резервирование
`reserve`
- длина строки
(без нуля)
`length`
`size`
- изменение длины
(дополняется пробелами
или усекается)
`resize`

Преобразование в С-строку

```
const Ch* c_str() const
```

– записывает символы в массив, дополняя их
финальным нулем

```
const Ch* data() const
```

– записывает символы в массив

- копирование из внутреннего буфера во
внешний
copy

Присоединение к строке

- `append`
- `operator+=`

пример:

```
strFI.append(strFamilyName) ;  
strFI.append(" ");  
strFI += strName;
```

Конкатенация строк

- объединение содержимого строк
`operator+`
- избегать массового использования

пример:

```
strFI = strFamilyName + string(" ")  
      + strName;
```

Вставка символов

- `insert` - вставка подстроки

пример:

```
strFI.insert(posName, name);
```

- `replace` - вставка с заменой

пример:

```
st.replace(pos, lnOld, lnNew, name);
```

— вставка начинается перед указанной позицией

Поиск в строке

- `string::npos` – при неудачном поиске
- `find`
 - заданная группа символов
- `rfind`
 - с конца, заданная группа символов

Поиск совпадающих

- `find_first_of`
 - первый символ из заданной группы
- `find_last_of`
 - последний символ из заданной группы

Поиск отличающихся

- `find_first_not_of`
 - первый символ не совпадающий ни с одним из символов заданной группы
- `find_last_not_of`
 - последний символ не совпадающий ни с одним из символов заданной группы

Доступ к символу

- `operator[]`
 - по индексу без контроля выхода за границы
- `at`
 - по индексу с контролем выхода за границы
 - при ошибке исключение
`std::out_of_range`

Удаление символов

- `erase` – удаление символов
- `clear` – очистка строки

пример:

```
st.clear();  
st.erase(lBeg, string::npos);  
st.erase(lBeg, lEnd - lBeg + 1);
```


Проверка на пустоту

```
bool empty() const
```

пример:

```
if (0 == str.empty()) {  
    // строка не пустая  
    //обработка  
}
```

ПОТОКОВЫЙ ВВОД И ВЫВОД

- **ВЫВОД**

`operator<<`

- **ВВОД**

`operator>>`

Сравнение символов

- лексикографическое сравнение
 - упорядочение в алфавите
 - упорядочение в кодовой таблице
- символы нижнего регистра меньше символов верхнего

Сравнение строк

- происходит лексикографическим сравнением символов до первого несовпадения
- упорядоченность строк определяется порядком несовпадающих символов

Функции сравнение строк

- `operator==`
- `operator!=`
- `operator>`
- `operator<`
- `operator>=`
- `operator<=`
- `compare`

Как лучше?

- с учетом специфики задачи
- UTF-8 внутри `std::string`
для «интерфейсов» с учетом 0-терминатора
- UTF для внутреннего представления

ваши вопросы

Какие основные недостатки
`std::string`?

Формирование строк (прямое сложение)

- **сложение**

```
std::string s = ssl ? "https" : "http";  
s = s + "://" + path + "/" + query;
```

- **ВЫВОД В ПОТОК (строковый)**

```
std::ostringstream ss;  
ss << (ssl ? "https" : "http") << "://" <<  
path << "/" << query;
```

Формирование строк (прямое сложение)

- форматирование

```
auto fmt = std::format("{}: // {} / {}",  
    (ssl ? "https" : "http"), path, query);
```

- готовность реализаций под вопросом
- «питонизм» на марше

Статические строки

```
static const std::string gId = "go";  
// .....  
int FuncGo(const std::string& arg);  
// .....  
FuncGo(gId);
```

Статические строки

```
static const char* gId = "FOO";  
// ...  
int FuncGo (const std::string& arg);  
// ...  
foo(gId);
```

Константы `std::string`

```
// h1.hpp
const std::string gRoot = "/usr/local/";

// h2.hpp
const std::string gPlug = gRoot + "h2.so";
```

Глобальные `std::string`

- инициализация требует памяти в куче
- инициализация происходит при выполнении программы
- порядок инициализации не определен между единицами трансляции
- брошенные при создании исключения не отлавливаются

Как «победить» недостатки
«простой» реализации
`std::string`?

Копирование при записи, COW

(Copy On Write, implicit sharing, shadowing)

- ресурс используется совместно для чтения (общая копия области данных)
- счетчик владельцев
- перед изменением ресурс копируется

примеры (см. Операционные Системы):

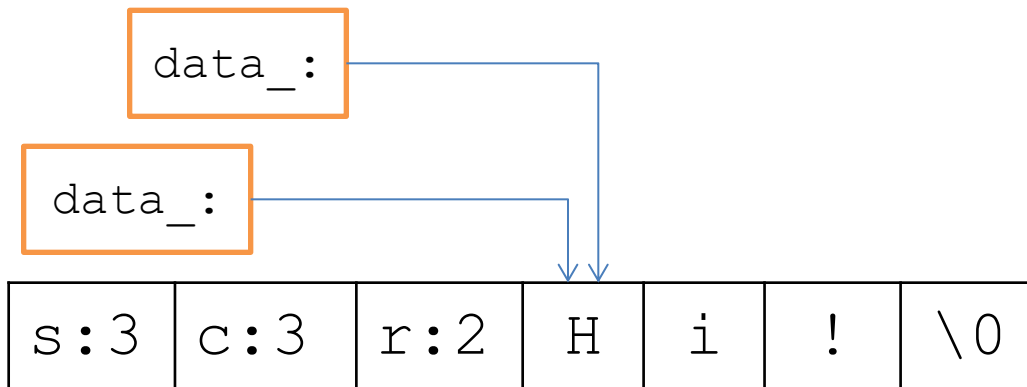
- оперативная память
- файлы на диске

COW для строк

```
struct StringBuf {  
    char* data;  
    size_t size;  
    size_t capacity;  
    int refcount;  
    // ...  
};  
  
class String {  
    StringBuf* buf_;  
    // ...  
};
```

COW для строк (GCC std::string, v < 5)

```
std::string s = "Hi ";  
const char *p = &a[3];  
a += "world";
```



Copy On Write

за

- экономия памяти
(на копии)
- легкое копирование
- уменьшение числа
аллокаций/удалений

против

- трата памяти
(реализация)
- лишний уровень
косвенности
- thread safety
(multithreaded COW
disease)
- инвалидация указателей

Инвалидация указателей

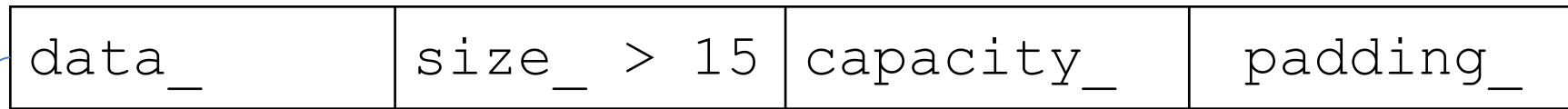
```
std::string s = "Hi!";  
const char* p = &a[1];  
s[1] = "u";
```

- в случае COW-строк указатели инвалидируются `operator[]` (даже для безопасных операций)
- мы не знаем кто владеет новым, а кто старым
- в C++11 запретили инвалидировать указатели при вызове `operator[]` (COW попал под запрет в std)

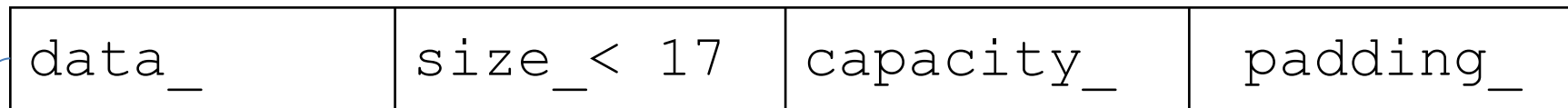
Оптимизация коротких строк, SSO (Small String Optimization)

- техника хранения коротких строк внутри экземпляра строки (а не в куче)
- простое копирование
- перемещение коротких строк теряет смысл

SSO для строк (GCC std::string, v>=5)



string in heap\0



Как «победить» недостатки
«простой» реализации
`std::string`?

`std::string_view`

- невладеющий «указатель на строку» с ссылочной семантикой (начало и конец)
- легко копируется
- не оканчивается `'\0'`
- использовать
 - для передачи параметров
 - с осторожностью возвращать и хранить

`std::string_view`

- **доступ на чтение**
 - `copy`, `substr`, `compare`, `find`
 - `data`
- **модификация**
 - `remove_prefix`
 - `remove_suffix`

std::string_view литералы

```
//using namespace std::string_view_literals;
```

```
auto u8str2 = u8"Hello World"sv;
```

```
auto str3 = L"hello"sv;
```

```
auto str4 = u"hello"sv;
```

```
auto str5 = U"hello"sv;
```

ваши вопросы